

**Universidad Tecnológica Nacional
Facultad Regional Mendoza.**

Tecnicatura Universitaria En Programación.



Trabajo integrador final
Programación 2.

Proyecto
Food-store.

Profesores: Hualpa Ramiro, Rigonni Cinthia.

Alumnos: Liendo Mateo, Avila Lucas, Pagano Amanda.

Comisión: 4.

Tabla de contenidos

Introducción	3
Marco teórico	4
Decisiones técnicas y arquitectura	5
Dificultades y Soluciones	7
Conclusión	8
Bibliografía	9

Introducción

El presente trabajo práctico integrador corresponde a la materia Programación 2 de la Tecnicatura Universitaria en Programación de la UTN. El proyecto consiste en el desarrollo de una aplicación de consola llamada Food Store, orientada a la gestión de pedidos de comida.

El sistema permite administrar las entidades principales del negocio — categorías, productos, usuarios y pedidos — mediante operaciones CRUD completas accesibles desde un menú interactivo por consola. A diferencia de un enfoque web, la interacción ocurre exclusivamente a través de la terminal, sin interfaz gráfica ni API REST.

El desarrollo integra los contenidos trabajados durante la cursada: Programación Orientada a Objetos aplicada al modelado de entidades con herencia e interfaces, el patrón DAO para separar la lógica de acceso a datos, y persistencia real mediante JDBC conectado a una base de datos MySQL. Se aplicaron además buenas prácticas de programación como validaciones defensivas, manejo de excepciones propias, transacciones para garantizar la integridad de los datos, y una arquitectura en capas clara y mantenible.

El proyecto fue desarrollado de forma grupal utilizando GitHub como herramienta de control de versiones, permitiendo el trabajo colaborativo y el seguimiento de los cambios realizados por cada integrante.

Marco Teórico aplicado al proyecto

Java 21

El proyecto fue desarrollado en Java 21, aprovechando características modernas del lenguaje como el operador `switch` con expresiones de flecha (`->`), el uso de `var` implícito en `try-with-resources`, y los `records` de texto. Se eligió esta versión por ser LTS y ser la requerida por la cátedra.

Programación Orientada a Objetos

El diseño del sistema refleja directamente los pilares de la POO:

- La clase abstracta `Base` centraliza los atributos comunes `id`, `eliminado` y `createdAt`, evitando repetición en las 5 entidades del sistema.
- `Categoria` tiene una relación 1:N con `Producto` — una categoría puede tener muchos productos pero cada producto pertenece a una sola categoría.
- `Usuario` tiene una relación 1:N con `Pedido` — un usuario puede tener muchos pedidos.
- `Pedido` tiene una relación de composición 1:N con `DetallePedido` — los detalles no existen sin el pedido.
- La interfaz `Calculable` obliga a `Pedido` a implementar `calcularTotal()`, que recorre la lista de detalles sumando cada subtotal.

JDBC

La conexión a la base de datos se centraliza en `ConexionBD`, que expone un único método estático `getConnection()`. Todos los DAOs obtienen su conexión desde ahí, evitando repetir credenciales. Se usó `PreparedStatement` en todas las operaciones SQL para evitar inyección, y `try-with-resources` para garantizar el cierre automático de conexiones.

Patrón DAO

Se implementó una interfaz genérica `GenericDAO<T>` con los 5 métodos CRUD base, que cada interfaz específica extiende agregando métodos propios. Por ejemplo, `UsuarioDAO` agrega `existeMail()` y `buscarPorMail()`, mientras que `PedidoDAO` agrega `listarPorUsuario()`. Las implementaciones concretas en `dao/impl` son el único lugar del proyecto donde existe código SQL.

Transacciones

Al crear un pedido, se usa `setAutoCommit(false)` para iniciar una transacción manual. Primero se inserta el pedido en la tabla `pedido`, luego cada `DetallePedido` en `detalle_pedido`. Si cualquier inserción falla, se ejecuta `rollback()` que revierte todo, dejando la base de datos en el estado anterior. Si todo sale bien, se ejecuta `commit()`.

Soft delete

Ninguna entidad se elimina físicamente de la base de datos. En su lugar, se actualiza el campo `eliminado = true` mediante un `UPDATE`. Todas las consultas `SELECT` filtran con

`WHERE eliminado = false`, por lo que los registros dados de baja no aparecen en los listados pero se conserva el historial completo.

Decisiones Técnicas y Arquitectura

Arquitectura por capas

El proyecto se organizó en capas con responsabilidades bien definidas:

- **entities:** contiene el modelo de dominio. Las clases representan las entidades del negocio y no contienen lógica de validación ni acceso a datos.
- **enums:** define los tipos enumerados del sistema (`Rol`, `Estado`, `FormaPago`), garantizando que solo se usen valores válidos.
- **interfaces:** contiene la interfaz `Calculable`, que define el comportamiento de cálculo de totales.
- **exception:** centraliza las excepciones propias del dominio (`DatoInvalidoException`, `EmailDuplicadoException`, `EntidadNoEncontradaException`, `StockInvalidoException`).
- **dao:** define los contratos de persistencia mediante interfaces (`GenericDAO<T>` y las específicas por entidad).
- **dao/Impl:** contiene las implementaciones JDBC de cada DAO, siendo el único lugar del proyecto donde se escribe SQL.
- **service:** coordina los casos de uso, aplica validaciones de negocio y delega la persistencia a los DAOs.
- **config:** centraliza la configuración de conexión a la base de datos en la clase `ConexionBD`.
- **menu:** gestiona la interacción con el usuario mediante submenús modulares, uno por entidad.

Interfaz genérica `GenericDAO<T>`

Se diseñó una interfaz genérica con los métodos CRUD comunes (`crear`, `buscarPorId`, `listar`, `editar`, `eliminar`) para evitar repetición de código. Cada interfaz específica la extiende y agrega métodos propios según las necesidades de cada entidad.

Soft delete (baja lógica)

En lugar de eliminar físicamente los registros de la base de datos, se optó por marcar el campo `eliminado = true`. Esto preserva el historial y evita romper la integridad referencial — por ejemplo, un pedido puede seguir mostrando sus detalles aunque el producto esté dado de baja.

Modularización del menú

Cada entidad tiene su propia clase de menú (`CategoriaMenu`, `ProductoMenu`, `UsuarioMenu`, `PedidoMenu`), lo que mantiene el `AppMenu` limpio y facilita el mantenimiento. Cada submenú incluye validaciones de entrada en tiempo real con reingreso automático ante errores.

Validaciones defensivas

Las validaciones se implementaron en dos niveles:

- **Service:** valida las reglas de negocio (precio no negativo, mail único, nombre mínimo 2 caracteres, etc.) antes de persistir.
- **Menú:** valida el formato de entrada del usuario (números válidos, letras donde corresponde, opción de cancelar con 0) para evitar que datos incorrectos lleguen al service.

Dificultades y Soluciones

Mezcla de proyectos en la misma carpeta

Al trabajar en equipo, inicialmente los proyectos de distintos integrantes quedaron mezclados en una misma carpeta, lo que generaba errores de compilación y confusión en IntelliJ. La solución fue organizar cada proyecto en su propia carpeta independiente y clonar el repositorio compartido de GitHub correctamente.

Error "Cannot resolve symbol AppMenu"

IntelliJ perdía el contexto del proyecto en algunas ocasiones, mostrando errores de importación aunque el código era correcto. La solución fue usar **File** → **Invalidate Caches** → **Invalidate and Restart**, que fuerza a IntelliJ a reindexar todo el proyecto.

Detalles del pedido no se persistían en la BD

Al crear un pedido, los detalles se agregaban correctamente en memoria pero no se guardaban en la tabla `detalle_pedido`. Esto ocurría porque el flujo de inserción en `PedidoDAOImpl` se ejecutaba antes de que el usuario agregara los detalles desde el menú. La solución fue agregar el método `insertarDetalle()` en `PedidoDAOImpl` y llamarlo desde `PedidoService.agregarDetalle()` cada vez que el usuario agrega un producto al pedido.

Paquete `service` vs `services`

Al integrar el trabajo de distintos integrantes del grupo, se detectó que uno usaba el paquete `service` y otro `services`. Esto generaba errores de importación. Se unificó todo bajo el nombre `service` para mantener la consistencia.

Importaciones duplicadas en IntelliJ

Al copiar y modificar código entre clases, algunos imports quedaron duplicados o sin usar, marcados en gris por IntelliJ. Se resolvió eliminando manualmente los imports innecesarios y usando **Alt + Enter** para que IntelliJ sugiriera las correcciones automáticamente.

Conclusión

Llegar al final de este proyecto nos deja con una sensación bastante particular: la de haber arrancado sin saber muy bien a dónde íbamos y terminar con algo que realmente funciona, se conecta a una base de datos y persiste información real.

Al principio, conceptos como DAO, JDBC o transacciones sonaban a palabras sueltas que habíamos visto en el campus, o en clases. Durante el desarrollo de este proyecto, esas palabras se convirtieron en código concreto que tuvimos que escribir, romper, entender y corregir. Entender por qué un detalle de pedido no se guardaba en la base de datos, o por qué IntelliJ perdía el contexto del proyecto, fueron problemas reales que nos obligaron a buscar soluciones reales.

El manejo de transacciones en la creación de pedidos fue uno de los desafíos más importantes del proyecto, ya que requirió comprender cómo JDBC gestiona el control de flujo ante fallos y cómo garantizar que la base de datos nunca quede en un estado inconsistente.

Trabajar en equipo con GitHub fue más familiar, un entorno ya conocido lo cual facilitó la comunicación y división de tareas. Coordinar cambios, hacer commits descriptivos y no pisarse el código entre integrantes.

En definitiva, Food Store no es solo un trabajo práctico entregado — es el proyecto donde todo lo que veníamos viendo de forma separada finalmente tomó forma en conjunto. La persistencia real, las transacciones, el patrón DAO y la arquitectura en capas dejaron de ser conceptos del campus para convertirse en decisiones concretas que tuvimos que tomar, justificar y sostener a lo largo del desarrollo. Y el resultado, creemos, lo demuestra.

Bibliografía / Webgrafía

- Documentación oficial de Java 21: <https://docs.oracle.com/en/java/javase/21/>
- Documentación de MySQL: <https://dev.mysql.com/doc/>
- Repositorio del proyecto: https://github.com/Mateows/Integrador_Programacion2
- Maven Repository (conector MySQL):
<https://mvnrepository.com/artifact/com.mysql/mysql-connector-j>
- Documentación de JDBC: <https://docs.oracle.com/javase/tutorial/jdbc/>